



ANÁLISIS TÉCNICO · AUDITORÍA · HOJA DE RUTA

Análisis, Auditoría de Seguridad y Mejoras

Sistema de Gestión Cafetería UIDE — Backend Spring Boot & Frontend React

PROYECTO	GestionUIDE
TIPO DE INFORME	Análisis del estado actual, brechas (gaps), auditoría de seguridad y plan de mejoras e innovación
MARCO DE REFERENCIA	OWASP Top 10 (2021), OWASP ASVS, buenas prácticas Spring Security y React
VERSIÓN	1.0
FECHA	Junio de 2026
CONFIDENCIALIDAD	Confidencial — contiene hallazgos de seguridad

Los hallazgos se basan en la lectura del código fuente entregado (análisis estático). No se realizaron pruebas de penetración sobre un entorno en ejecución. Las severidades son una estimación orientativa para priorizar la remediación.

Tabla de contenidos

- 1. Resumen ejecutivo**
- 2. Metodología y alcance de la evaluación**
- 3. Fortalezas del sistema actual**
- 4. Brechas funcionales (qué falta por hacer)**
- 5. Brechas técnicas y de arquitectura**
- 6. Auditoría de seguridad**
 - 6.1 Resumen de hallazgos
 - 6.2 Hallazgos críticos
 - 6.3 Hallazgos altos
 - 6.4 Hallazgos medios
 - 6.5 Hallazgos bajos / informativos
- 7. Calidad de código y mantenibilidad**
- 8. Hoja de ruta priorizada**
- 9. Ideas de innovación**
- 10. Conclusiones y recomendaciones**

1. Resumen ejecutivo

El sistema es un producto **funcionalmente completo y bien estructurado** para una cafetería: cubre el ciclo de pedido a cobro, incorpora roles, autenticación con JWT y Google, facturación con PDF y un panel de métricas. La arquitectura por capas y la organización del frontend son sólidas y facilitan el mantenimiento.

Sin embargo, la evaluación identifica un **riesgo de seguridad alto y concentrado en el control de acceso**: el backend autentica a los usuarios pero **no verifica su rol en ninguno de los endpoints**. En la práctica, cualquier usuario con una sesión válida (por ejemplo, un cliente) puede invocar operaciones de administrador —incluido asignarse a sí mismo el rol ADMIN—. A esto se suman credenciales sensibles incrustadas en el código, una configuración de CORS permisiva y la posibilidad de manipular pagos. Estos puntos deben resolverse **antes de cualquier puesta en producción**.

Dimensión	Estado	Síntesis
Funcionalidad	Buena	Cobertura amplia del dominio; faltan piezas de producción (pagos reales, tiempo real, paginación).
Arquitectura	Buena	Separación en capas clara; uso correcto de DTOs y manejo global de errores.
Seguridad	Crítica	Sin autorización por rol; secretos en el repositorio; CORS y pagos explotables.
Calidad de código	Media	Legible pero con duplicación de reglas, uso de Double para dinero y dependencias obsoletas.
Preparación para producción	Baja	Falta configuración por entornos, observabilidad, CI/CD y endurecimiento.

Prioridad inmediata (top 3)

1) Implementar autorización por rol en el servidor y bloquear la auto-asignación de roles. 2) Rotar y externalizar todas las credenciales que hoy están en el repositorio (clave JWT, contraseña de correo, base de datos). 3) Endurecer el módulo de pagos para que un cliente no pueda marcar facturas como pagadas.

2. Metodología y alcance de la evaluación

La evaluación se realizó mediante **análisis estático** del código fuente del backend (Spring Boot) y del frontend (React/TypeScript) entregados. Se revisaron: configuración de seguridad, filtro JWT, controladores REST, servicios de negocio, entidades, DTOs y la capa de servicios del frontend.

Elemento	Detalle
Marco de seguridad	OWASP Top 10 (2021) como guía de categorización; referencias a OWASP ASVS y buenas prácticas de Spring Security.
Escala de severidad	Crítica · Alta · Media · Baja · Informativa (según impacto y facilidad de explotación).
Limitaciones	No se incluyó pom.xml/build.gradle ni todas las clases de servicio; no hubo pruebas dinámicas. Algunos hallazgos podrían requerir confirmación en ejecución.

3. Fortalezas del sistema actual

Es importante reconocer lo que está bien resuelto, porque constituye una base sobre la que conviene construir:

Aspecto	Por qué es valioso
Arquitectura en capas	Separación limpia entre presentación, DTO, servicio, repositorio y modelo: facilita pruebas y evolución.
Contraseñas con BCrypt	Hash con sal por usuario; el campo se oculta con @JsonIgnore y nunca se expone en las respuestas.
Validación de entrada (DTO)	Uso de Bean Validation (@NotBlank, @Email, @Size, @Positive) en los datos de entrada.

Manejo global de errores	@RestControllerAdvice centraliza las respuestas de error con códigos HTTP coherentes.
Máquina de estados del pedido	Transiciones validadas y restauración de stock al cancelar; reduce inconsistencias.
Recuperación de contraseña discreta	No revela si un correo existe (buena práctica anti-enumeración).
OAuth2 con verificación de token	El <i>id token</i> de Google se verifica en el servidor, no solo en el cliente.
Frontend ordenado	Servicios por recurso, interceptores de Axios (token + manejo de 401), guardias por rol, modo oscuro y diseño responsivo.

4. Brechas funcionales (qué falta por hacer)

Funcionalidades que un sistema de cafetería en producción normalmente necesita y que hoy no están implementadas o están incompletas. Cada una incluye una recomendación accionable.

Brecha	Prioridad	Recomendación
Pasarela de pago real	Alta	El registro de pago es solo contable. Integrar una pasarela (en Ecuador: PayPhone, Datafast, Kushki, DeUna) con confirmación del proveedor antes de marcar la factura como pagada.
Facturación electrónica (SRI)	Alta	Para uso real en Ecuador, emitir comprobantes electrónicos autorizados por el SRI (XML firmado + RIDE). Hoy solo se genera un PDF interno.
Notificaciones en tiempo real	Media-Alta	Cocina y clientes no reciben actualizaciones automáticas. Añadir WebSocket/SSE para enviar pedidos a cocina y reflejar el estado en vivo.
Paginación y filtros en listados	Media-Alta	Los endpoints de listado devuelven todos los registros. Implementar Pageable (page/size/sort) para escalar.
Verificación de correo	Media	El registro solo envía bienvenida. Agregar verificación de correo (doble opt-in) para reducir cuentas falsas.
Tokens de refresco / sesión	Media	El token dura 24 h sin renovación ni revocación. Añadir <i>refresh tokens</i> y una lista de revocación para el logout real.
Persistencia del código de recuperación	Media	Los códigos viven en memoria (se pierden al reiniciar y no funcionan en varias instancias). Persistir con expiración y límite de intentos.
Control de conflictos de reservas	Media	Validar solapamiento de horarios y capacidad de mesa al confirmar una reserva; recordatorios por correo.
Auditoría de acciones	Media	No hay bitácora de operaciones sensibles (cambios de rol, pagos, eliminaciones). Registrar quién hizo qué y cuándo.
Documentación de API viva	Baja	Incorporar springdoc/OpenAPI para exponer la API en Swagger UI y mantenerla sincronizada con el código.
Reportes exportables	Baja	Exportar ventas y métricas a Excel/CSV/PDF desde el panel.

5. Brechas técnicas y de arquitectura

Brecha	Prioridad	Recomendación
Configuración por entornos	Alta	Existe un único <code>application.properties</code> . Separar perfiles (dev, prod) y mover secretos a variables de entorno / gestor de secretos.
Esquema gestionado por Hibernate	Alta	<code>ddl-auto=update</code> es peligroso en producción (cambios de esquema no controlados). Migrar a Flyway o Liquibase con scripts versionados.
Dinero con Double	Alta	Subtotales, IVA, totales y montos usan punto flotante: produce errores de redondeo en valores monetarios. Migrar a <code>BigDecimal</code> con escala fija.
Condición de carrera en stock	Alta	Se verifica el stock y luego se descuenta en pasos separados, sin bloqueo. Bajo concurrencia se puede sobrevender. Usar decremento atómico o bloqueo optimista (<code>@Version</code>).
Dependencias obsoletas	Media	Spring Boot 2.x (javax.*), <code>WebSecurityConfigurerAdapter</code> (eliminado en Spring Security 6) y API antigua de jwt. Planificar actualización a Spring Boot 3.x para seguir recibiendo parches de seguridad.
Regla de IVA duplicada	Media	El 15% está “quemado” en el frontend y en el cálculo de pedidos, mientras la facturación manual usa el IVA de la empresa. Centralizar la regla en una única fuente de verdad.
Sin pruebas de autorización	Media	Hay pruebas de integración, pero ninguna verifica el control de acceso por rol. Añadir pruebas que confirmen que un cliente no accede a endpoints de admin.
Observabilidad	Media	Logging mínimo y sin métricas/trazas. Incorporar Actuator, métricas (Micrometer/Prometheus) y trazabilidad de peticiones.
CI/CD y empaquetado	Baja	No hay pipeline ni contenedores. Añadir Dockerfile, docker-compose (app + MySQL) y un

flujo de integración con análisis estático (SAST) y escaneo de dependencias.

Frontend: robustez

Baja

Añadir *error boundaries*, manejo de carga consistente, renovación de token y pruebas (Vitest/RTL). Evaluar PWA para uso en tablets del local.

Sobre la precisión monetaria

Con `double`, operaciones como $0.1 + 0.2$ no dan exactamente 0.3 . En facturación esto se traduce en centavos que no cuadran y en totales que difieren entre frontend y backend. `BigDecimal` con redondeo definido (por ejemplo, `HALF_UP` a 2 decimales) elimina el problema.

6. Auditoría de seguridad

6.1 Resumen de hallazgos

ID	Hallazgo	Severidad	OWASP 2021
SEC-01	Ausencia de autorización por rol en el backend	Crítica	A01 Broken Access Control
SEC-02	Escalada de privilegios vía endpoints de usuarios/roles	Crítica	A01 Broken Access Control
SEC-03	Credenciales y secretos incrustados en el repositorio	Crítica	A05 / A07
SEC-04	Manipulación de pagos y estado de factura	Crítica	A04 Insecure Design
SEC-05	CORS permisivo con credenciales	Alta	A05 Security Misconfiguration
SEC-06	Referencia directa insegura a objetos (IDOR)	Alta	A01 Broken Access Control
SEC-07	Subida de archivos sin restricciones / path traversal	Alta	A03 / A04
SEC-08	Debilidades en la gestión del token JWT	Alta	A02 / A07
SEC-09	Divulgación de información (errores y enumeración en login)	Media	A05 / A07
SEC-10	Sin limitación de tasa ni protección de fuerza bruta	Media	A07 Auth Failures
SEC-11	Código de recuperación débil y volátil	Media	A04 / A07
SEC-12	Seguridad de transporte sin reforzar	Media	A02 Cryptographic Failures
SEC-13	Token almacenado en localStorage (robo vía XSS)	Media	A05 / A03
SEC-14	Gestión de esquema y configuración sin endurecer	Media	A05 Security Misconfiguration
SEC-15	Componentes desactualizados	Baja	A06 Vulnerable Components
SEC-16	Sin registro de auditoría de seguridad	Baja	A09 Logging Failures
SEC-17	Control de rol del frontend solo cosmético	Info	A01 (refuerza SEC-01)

6.2 Hallazgos críticos

SEC-01 — Ausencia de autorización por rol en el backend Crítica

OWASP

A01:2021 — Broken Access Control

DESCRIPCIÓN

La configuración de seguridad solo exige que el usuario *esté autenticado* para `/api/**`. No existe ninguna verificación de rol: no se usan `@PreAuthorize`, `@Secured` ni reglas `hasRole(...)`, y la seguridad por método no está habilitada. El rol viaja en el token y se usa para construir la interfaz, pero el servidor nunca lo comprueba al ejecutar la operación.

EVIDENCIA

`WebSecurityConfig: .antMatchers("/api/**").authenticated()`. Búsqueda de anotaciones de autorización en todo el backend: 0 coincidencias.

IMPACTO

Cualquier usuario con sesión válida (p. ej. un CLIENTE) puede invocar operaciones de cualquier rol: listar todos los usuarios, eliminar productos, cambiar precios, ver todas las facturas, etc. Es la raíz de varios hallazgos posteriores.

REMEDIACIÓN

Habilitar la seguridad por método (`@EnableMethodSecurity`) y anotar cada endpoint con el rol requerido (p. ej. `@PreAuthorize("hasRole('ADMIN')")`), o declarar reglas por ruta y método en la configuración (`.antMatchers(POST, "/api/productos").hasRole("ADMIN")`). Aplicar el principio de mínimo privilegio en todos los recursos.

```
// Estado actual: solo exige sesión
.antMatchers("/api/**").authenticated()

// Recomendado (ejemplo)
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public ResponseEntity<?> eliminar(@PathVariable Long id) { ... }
```

SEC-02 — Escalada de privilegios vía endpoints de usuarios/roles Crítica

OWASP

A01:2021 — Broken Access Control

DESCRIPCIÓN

Como consecuencia de SEC-01, los endpoints de gestión de usuarios aceptan un rol arbitrario y no validan quién los llama. Un cliente puede crear un usuario ADMIN o asignarse el rol ADMIN a sí mismo.

EVIDENCIA

`POST /api/usuarios` (acepta rol en el cuerpo) y `PUT /api/usuarios/{id}/rol?rol=ADMIN`, ambos sin verificación de autorización.

IMPACTO

Toma de control total del sistema por parte de cualquier usuario registrado. Es el peor escenario: convierte una cuenta de cliente en super-administrador.

REMEDIACIÓN

Restringir estos endpoints exclusivamente a ADMIN; impedir que un usuario modifique sus propios roles; validar el rol entrante contra una lista permitida y registrar el cambio en auditoría (ver SEC-16).

SEC-03 — Credenciales y secretos incrustados en el repositorio Crítica

OWASP

A05 Security Misconfiguration / A07 Identification & Authentication Failures

DESCRIPCIÓN

El archivo de configuración contiene secretos en texto plano: clave de firma JWT, usuario de base de datos (root) con contraseña vacía, y usuario + contraseña de aplicación del correo SMTP. Además, la clave JWT aparece "quemada" como valor por defecto dentro del código del filtro.

EVIDENCIA

`jwt.secret=...`, `spring.datasource.username=root` con contraseña vacía, `spring.mail.password=...` en `application.properties`; y la constante `KEY_APP` con la misma clave por defecto en `JWTAuthorizationFilter`.

IMPACTO

Quien tenga acceso al repositorio puede firmar tokens válidos (suplantar a cualquier usuario), acceder a la base de datos y usar la cuenta de correo. La contraseña de aplicación de correo expuesta debe considerarse comprometida.

REMEDIACIÓN

Rotar de inmediato todos los secretos expuestos (revocar la contraseña de aplicación de Gmail, cambiar la clave JWT, poner credenciales reales a la base de datos). Externalizar a variables de entorno o a un gestor de secretos. Generar la clave JWT como una clave aleatoria larga (256 bits) y eliminar el valor por defecto del código. Añadir el archivo de configuración real a `.gitignore` y purgar el historial.

SEC-04 — Manipulación de pagos y estado de factura

Crítica

OWASP	A04:2021 — Insecure Design
DESCRIPCIÓN	El registro de pago acepta el facturaId y el monto directamente del cliente y, al alcanzar el total, marca la factura como PAGADA. No hay validación contra una pasarela ni verificación de quién paga. Combinado con SEC-01, un usuario puede registrar un pago por el total y dar por pagada una factura sin cobro real.
EVIDENCIA	POST /api/pagos, /pagos/efectivo?monto=, /pagos/tarjeta?monto=; en PagoService.registrar se cambia el estado de la factura si el total pagado cubre el total, sin confirmación externa.
IMPACTO	Pérdida económica directa: facturas marcadas como pagadas sin ingreso real; el pago “tarjeta” solo guarda una referencia de texto. También permite registrar pagos sobre facturas ajenas.
REMEDIACIÓN	Restringir el registro de pagos a CAJERO/ADMIN; para tarjeta, exigir confirmación de la pasarela antes de marcar como pagada; añadir idempotencia (evitar pagos duplicados) y validar que el monto no exceda el pendiente. Registrar cada pago en auditoría.

6.3 Hallazgos altos

SEC-05 — CORS permisivo con credenciales Alta

OWASP	A05 Security Misconfiguration
DESCRIPCIÓN	El backend permite cualquier origen (<code>allowedOriginPatterns("*")</code>) y a la vez habilita credenciales (<code>allowCredentials(true)</code>). Además, cada controlador repite <code>@CrossOrigin(origins="*")</code> .
IMPACTO	Refleja cualquier origen con credenciales, lo que facilita ataques desde sitios de terceros. La combinación “todos los orígenes + credenciales” es una configuración explícitamente desaconsejada.
REMEDIACIÓN	Definir una lista blanca de orígenes (dominios del frontend) en lugar de comodín; eliminar los <code>@CrossOrigin("*")</code> de los controladores y centralizar la política CORS.

SEC-06 — Referencia directa insegura a objetos (IDOR) Alta

OWASP	A01 Broken Access Control
DESCRIPCIÓN	Varios endpoints reciben el identificador de la entidad en la ruta y no verifican que el recurso pertenezca al usuario que lo solicita.
EVIDENCIA	<code>GET /api/pedidos/cliente/{id}</code> , <code>GET /api/facturas/cliente/{id}</code> , <code>/api/favoritos/usuario/{id}</code> , <code>PUT /api/usuarios/{id}/perfil</code> : cambiando el <code>id</code> se accede a datos de otros usuarios.
IMPACTO	Un cliente puede leer pedidos, facturas y favoritos de otros clientes, o modificar el perfil de otra persona (fuga de datos personales).
REMEDIACIÓN	Derivar la identidad del usuario del token (no de un parámetro) y validar la propiedad del recurso antes de devolver o modificar datos; permitir el acceso transversal solo a ADMIN.

SEC-07 — Subida de archivos sin restricciones / path traversal Alta

OWASP	A03 Injection / A04 Insecure Design
DESCRIPCIÓN	Las subidas de imagen de producto y foto de usuario construyen la ruta del archivo concatenando directamente el nombre original enviado por el cliente, sin validar tipo, extensión ni contenido. Los archivos se sirven públicamente bajo <code>/uploads/**</code> .
EVIDENCIA	<code>UsuarioWS.subirFoto / ProductoWS.subirImagen: System.currentTimeMillis() + "_" + archivo.getOriginalFilename()</code> usado como nombre/ruta.
IMPACTO	Un nombre con <code>../</code> podría escribir fuera del directorio previsto; subir HTML/SVG malicioso servido desde el mismo origen abre la puerta a XSS almacenado. Además, estos endpoints no exigen rol.
REMEDIACIÓN	Generar el nombre del archivo en el servidor (UUID), validar tipo MIME y extensión contra una lista blanca de imágenes, verificar la firma del archivo, limitar tamaño (ya hay límite de 5 MB) y servir las descargas con <code>Content-Disposition</code> y cabeceras que impidan la ejecución.

SEC-08 — Debilidades en la gestión del token JWT Alta

OWASP	A02 Cryptographic Failures / A07 Auth Failures
DESCRIPCIÓN	Clave simétrica corta y fija (compartida y con valor por defecto en el código), sin rotación; el cierre de sesión no invalida el token (logout devuelve OK sin revocar); expiración de 24 h sin refresco; sin validación de emisor/audiencia; uso de la API antigua de <code>jjwt</code> .
IMPACTO	Un token robado es válido hasta 24 h y no puede revocarse; una clave débil/filtrada permite forjar tokens (relacionado con SEC-03).
REMEDIACIÓN	Clave aleatoria de 256 bits desde el entorno; tokens de acceso cortos + refresh token; lista de revocación o versión de token por usuario; validar emisor/audiencia; actualizar a la API moderna de <code>jjwt</code> .

6.4 Hallazgos medios

SEC-09 — Divulgación de información Media

DESCRIPCIÓN	El manejador genérico devuelve <code>"Error interno: " + ex.getMessage()</code> al cliente (puede filtrar detalles internos / SQL). En el login, los mensajes distinguen entre “usuario no encontrado” y “contraseña incorrecta”, lo que permite enumerar cuentas.
REMEDIACIÓN	Responder mensajes genéricos al cliente y registrar el detalle solo en logs del servidor; usar un mensaje único e indistinto para credenciales inválidas en el login.

SEC-10 — Sin limitación de tasa ni protección de fuerza bruta Media

DESCRIPCIÓN	Los endpoints de login, recuperación y verificación de código no tienen límite de intentos ni bloqueo temporal.
REMEDIACIÓN	Añadir limitación de tasa (por IP/cuenta), bloqueo progresivo y, opcionalmente, CAPTCHA tras varios fallos.

SEC-11 — Código de recuperación débil y volátil Media

DESCRIPCIÓN	El código de 6 dígitos se genera con Random (no criptográfico), se guarda en memoria (se pierde al reiniciar y no sirve en varias instancias) y no limita el número de intentos de verificación.
REMEDIACIÓN	Usar SecureRandom, persistir el código con expiración, limitar intentos e invalidar tras el primer uso.

SEC-12 — Seguridad de transporte sin reforzar Media

DESCRIPCIÓN	La conexión a MySQL usa useSSL=false y la aplicación expone HTTP en el puerto 8080. Aunque se envían cabeceras HSTS, no hay TLS forzado a nivel de aplicación.
REMEDIACIÓN	Servir siempre tras HTTPS (proxy inverso con certificado), habilitar TLS hacia la base de datos y redirigir HTTP→HTTPS.

ID	Hallazgo	Síntesis y remediación
SEC-13	Token en localStorage Media	El JWT se guarda en localStorage, accesible por JavaScript: un XSS podría robarlo. Considerar cookie HttpOnly + protección CSRF, o tokens de vida corta y endurecer el frontend frente a XSS.
SEC-14	Esquema/config sin endurecer Media	ddl -auto=update en producción y un único archivo de configuración. Migrar a Flyway/Liquibase y separar perfiles dev/prod (ver sección 5).

6.5 Hallazgos bajos / informativos

ID	Hallazgo	Síntesis y remediación
SEC-15	Componentes desactualizados Baja	Spring Boot 2.x, WebSecurityConfigurerAdapter (retirado) y jjwt antiguo. Planificar la actualización para seguir recibiendo parches de seguridad.
SEC-16	Sin auditoría de seguridad Baja	No se registran acciones sensibles (cambios de rol, pagos, borrados). Añadir bitácora con actor, acción, recurso y marca de tiempo.
SEC-17	Control de rol cosmético en el frontend Info	Las guardias del frontend solo ocultan la interfaz; no son una medida de seguridad. La protección real debe estar en el servidor (SEC-01). Mantenerlas como mejora de experiencia, no como control.

Cadena de explotación más peligrosa

SEC-01 (sin autorización) habilita SEC-02 (un cliente se vuelve ADMIN) y SEC-04 (marcar facturas como pagadas). Resolver SEC-01 reduce drásticamente la superficie de ataque, por lo que debe ser la primera acción.

7. Calidad de código y mantenibilidad

Más allá de la seguridad, estos puntos afectan la robustez y la facilidad de evolución del sistema.

Aspecto	Valoración	Observación
Legibilidad y estructura	Buena	Nombres claros, paquetes coherentes y uso de Lombok para reducir el código repetitivo.
Consistencia de la API	Buena	Sobre de respuesta uniforme (ApiResponse) y manejo de errores centralizado.
Tipos monetarios	Mejorable	Uso de Double en dinero: riesgo de redondeo. Migrar a BigDecimal.
Duplicación de reglas	Mejorable	El IVA (15%) se repite en frontend y backend; centralizar en una única fuente.
Concurrencia	Mejorable	Control de stock sin bloqueo: posible sobreventa. Añadir bloqueo optimista (@Version).
Cobertura de pruebas	Mejorable	Existen pruebas de integración, pero ninguna valida autorización por rol ni reglas monetarias.
Gestión de dependencias	Mejorable	Versiones antiguas del framework; conviene un escaneo periódico de vulnerabilidades (SCA).

8. Hoja de ruta priorizada

Plan sugerido en tres fases. El esfuerzo es una estimación relativa (S = bajo, M = medio, L = alto).

Fase 1 — Endurecimiento de seguridad (antes de producción)

Acción	Impacto	Esfuerzo	Ref.
Implementar autorización por rol en el servidor	Muy alto	M	SEC-01
Bloquear auto-asignación de roles; restringir gestión de usuarios a ADMIN	Muy alto	S	SEC-02
Rotar y externalizar todos los secretos	Muy alto	S	SEC-03
Endurecer pagos (rol, idempotencia, confirmación)	Muy alto	M	SEC-04
Restringir CORS a orígenes conocidos	Alto	S	SEC-05
Validar propiedad de recursos (IDOR)	Alto	M	SEC-06
Asegurar subida de archivos	Alto	M	SEC-07

Fase 2 — Solidez técnica y operación

Acción	Impacto	Esfuerzo	Ref.
Migrar dinero a BigDecimal y centralizar el IVA	Alto	M	\$5
Migraciones de esquema (Flyway/Liquibase) y perfiles dev/prod	Alto	M	SEC-14
Bloqueo optimista para stock	Alto	S	\$5
Refresh tokens + revocación + límite de tasa	Medio	M	SEC-08/10
Paginación en listados	Medio	S	\$4
Observabilidad (Actuator/métricas) y auditoría	Medio	M	SEC-16
CI/CD + Docker + escaneo de dependencias	Medio	M	\$5

Fase 3 — Producto y crecimiento

Acción	Impacto	Esfuerzo	Ref.
Pasarela de pago real (PayPhone/Datafast/Kushki/DeUna)	Alto	L	\$4
Facturación electrónica SRI	Alto	L	\$4

Tiempo real a cocina (WebSocket/SSE)	Medio	M	\$4
Actualización a Spring Boot 3.x	Medio	L	SEC-15

9. Ideas de innovación

Oportunidades para diferenciar el producto, aprovechando que varias piezas (mesas, menú público, entidad empresa, métricas por mesero) ya existen.

Idea	Descripción y valor
Pedido por QR en la mesa	Un código QR por mesa abre el menú público y permite pedir desde el teléfono sin app. El sistema ya tiene mesas y menú: es una extensión natural de alto impacto.
Pantalla de cocina (KDS) en vivo	Tablero en tiempo real para cocina con los pedidos entrantes y su estado; el cliente ve el progreso de su pedido. Reduce errores y tiempos.
Inteligencia de demanda	Predicción de demanda por franja horaria y día para optimizar compras de stock y reducir desperdicio; recomendaciones de menú según historial.
Programa de fidelización	Puntos, cupones y promociones personalizadas usando el historial de pedidos y favoritos ya disponibles.
Analítica avanzada	Ampliar el panel: horas pico, ticket promedio, productos que se compran juntos y rendimiento por mesero (ya hay una base de ventas por mesero).
Multi-sucursal	La entidad Empresa habilita evolucionar a varias sucursales con catálogo y reportes por local.
App móvil / PWA	Versión instalable para clientes y para el personal en tablets, con soporte offline básico.
Pagos por QR / transferencia	Integrar pagos móviles (p. ej. DeUna) y conciliación automática con las facturas.
Reservas inteligentes y delivery	Chatbot para reservas, recordatorios automáticos e integración con plataformas de entrega (p. ej. PedidosYa).

10. Conclusiones y recomendaciones

El Sistema de Gestión Cafetería UIDE es una base **funcional y bien organizada**, con decisiones acertadas en arquitectura, validación de entrada, hashing de contraseñas y experiencia de usuario. Su principal limitación no es la funcionalidad sino el **control de acceso del lado del servidor**: hoy la autorización por rol no existe, lo que expone operaciones sensibles a cualquier usuario autenticado y habilita escalada de privilegios y manipulación de pagos.

La recomendación es clara: **tratar la Fase 1 como requisito bloqueante** antes de cualquier despliegue real. Con esas correcciones —autorización por rol, rotación de secretos, endurecimiento de pagos, CORS, IDOR y subidas— el sistema pasa de “prototipo funcional” a “candidato a producción”. Las Fases 2 y 3 consolidan la solidez técnica y abren camino a las innovaciones que diferencian al producto.

Mensaje final

El trabajo realizado es una plataforma sólida sobre la que vale la pena construir. Priorizando la seguridad del control de acceso y la gestión de secretos, el proyecto queda en muy buena posición para crecer de forma segura y sostenible.